

Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics

Anonymous Author(s)

Anonymous Institution
anonymous@example.org

Abstract. AWS Step Functions workflows are executable JSON artifacts, yet existing validators mostly check syntax and structural well-formedness. Data-flow mistakes, unsafe retries, missing compensations, parallel write races, and unbounded callbacks can therefore deploy and fail only at run time. We present STEP CHECK, a static verifier for Step Functions. Its core is a flow-sensitive analysis of the ASL I/O pipeline that reports a missing-field access only when every execution reaching the read would fail, giving no false positives under the modeled JSONPath semantics. Around this native core, seven further checks cover structural correctness, typed contracts, typestate ordering, retry safety, Saga compensation, conservative concurrency, and temporal bounds. Native ASL facts run immediately; omitted task semantics can be declared or inferred, with severity graded by provenance. On 193 real AWS workflows and 66 CNCF Serverless Workflow examples, STEP CHECK rediscovers six independently fixed real bugs, flags 33 unbounded callbacks, achieves 100% per-operator recall on 616 targeted mutants in six classes, and detects 67% (84/126) of injected missing-field reads, all oracle-confirmed, at tens of microseconds per workflow.

Keywords: Serverless computing · AWS Step Functions · Workflow verification · Data-flow analysis · Saga transactions

1 Introduction

Serverless applications increasingly encode business processes as managed workflows. In AWS Step Functions, the executable artifact is an Amazon States Language (ASL) state machine: JSON for task calls, branches, retries, parallelism, and error handling [3]. These workflows charge cards, reserve inventory, move documents, and coordinate human approvals, so latent workflow bugs become production failures rather than local exceptions.

Figure 1 shows one such workflow. It passes AWS’s syntactic validation, yet six latent defect classes survive. **(1)** `ChargeCard` reads a missing `$.total`; **(2)** reordering a single edge would let it ship before payment; and **(3)** a broad retry can double-charge. The same workflow also shows **(4)** two parallel branches

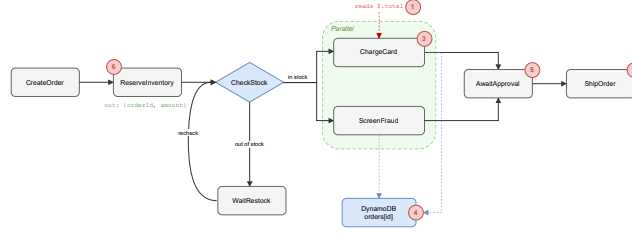


Fig. 1. An order-processing workflow that passes AWS’s syntactic validation yet hides six latent defect classes; marker **(2)** is a one-edge ship-before-pay mutation, not an existing edge.

updating the same item, **(5)** a callback that can wait forever, and **(6)** a committed reservation without compensation (Section 2.2).

Existing validators check that the JSON is well-formed, transitions exist, and paths are syntactic [4,7,2]. They do not know whether a task produces the field its successor reads, whether a retry is safe for a non-idempotent effect, whether two branches race on one resource, or whether a durable effect is compensated after failure. The missing facts are semantic: task contracts, protocol states, idempotency, write footprints, time bounds, and compensators. Durable-execution systems enforce recovery at run time [8], not before deployment.

We present STEP CHECK, a static verifier for AWS Step Functions. Its thesis is simple: verify the *deployed artifact* under *layered semantics*. The unifying contribution is a single workflow IR: its native facts support sound or deliberately under-approximating checks, and the business facts ASL omits are recovered as provenance-labelled premises. Native facts in ASL support immediate structural, data-flow, temporal, and conservative concurrency checks. Facts ASL omits can arrive from a sidecar, typed DSL, generated metadata, or lightweight inference; STEP CHECK grades findings by that provenance, so inferred facts produce warnings while declared facts produce errors relative to the declared model. Adoption therefore starts on raw ASL and strengthens as teams add semantics.

The core technical result is a flow-sensitive JSON-shape analysis for ASL’s I/O pipeline: it reports a missing-field access only when every execution reaching the read lacks the field (Theorem 1). Around it, seven further passes cover structural correctness, typed contracts, typestate ordering, retry safety, Saga compensation, conservative concurrency, and temporal bounds.

Contributions.

- A deployed-artifact verification architecture for ASL with native and declared semantic layers.
- A no-false-positive data-flow analysis for missing JSON fields (under the modeled JSONPath semantics), plus workflow checks for retry, ordering, concurrency, time, and compensation.

- A Rust implementation with raw ASL, typed-DSL, and CNCF Serverless Workflow front ends.
- Evidence on 193 AWS and 66 CNCF workflows, plus 95 from independent repositories: six real bugs rediscovered, 84 oracle-confirmed missing-field detections with zero false positives, and 100% recall on 616 targeted mutants produced by STEPCheck’s mutator.

2 Background and Problem Setting

ASL exposes enough structure for native checks, but hides the business facts needed for deeper workflow verification. This section fixes the small slice of ASL semantics used by the analyses, then states the omitted facts and design requirements.

2.1 The Amazon States Language as a verification target

A Step Functions workflow is a state machine with a **StartAt** state and named states of eight kinds: **Task**, **Choice**, **Parallel**, **Map**, **Wait**, **Pass**, **Succeed**, and **Fail**. A single JSON document is threaded through the graph. Each state may reshape it through the fixed ASL I/O pipeline: **InputPath** selects input; **Parameters** or **ItemSelector** constructs a task payload; **ResultSelector** reshapes the raw result; **ResultPath** merges that result back into the raw input; and **OutputPath** selects the successor document. A key ending in **.\$** reads a JSONPath from the current document (e.g. a **"Payload.\$": "\$.detail.customer"** constructor reads **\$.detail.customer**), and the runtime fails the reference when that path is absent. Current ASL also has workflow variables and JSONata: **Assign** stores **\$var** values for later states, and JSONata fields read **\$states.input**, **\$states.result**, and workflow variables. STEPCheck tracks variables and modeled JSONata object/direct-reference fragments; arbitrary expressions collapse to **T**. Fault handling is also explicit: **Retry** re-executes a state on matching errors, **Catch** routes an error to a handler, and **TimeoutSeconds/HeartbeatSeconds** bound long-running tasks and callbacks.

2.2 What Asl omits, and why the defects are latent

Figure 1 uses a standard order workflow with create, reserve, charge, and ship tasks. The intended protocol is created $\rightarrow$ reserved $\rightarrow$ paid $\rightarrow$ shipped. ASL records the graph and JSON reshaping. The defect classes in this paper require four semantic facts that ASL omits: a task contract $T : A \rightarrow B$ over input/output fields; a typestate edge $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ in a business protocol δ ; idempotency of external effects; and persistence plus a compensator for durable effects.

Because validators inspect only syntax and structural well-formedness, all six defects deploy: each is valid JSON over a valid graph, and the platform retries and routes errors but infers no contract, business protocol, idempotency, write footprint, or Saga intent.

2.3 Design requirements for an in-place verifier

An in-place verifier must therefore satisfy four requirements. **(R1) In place:** check the deployed ASL artifact at build time, without a new runtime. **(R2) Sound where native:** when a property follows from ASL alone, report only definite faults. **(R3) Semantic premises:** accept omitted facts from sidecars, DSLs, generated metadata, or inference, and grade severity by provenance. **(R4) One framework:** cover data-flow, ordering, retry, concurrency, temporal, and compensation defects in one tool. In summary, control flow and JSONPath movement are native, sound passes. Contracts, typestate, idempotency, persistence, and compensators are declared or inferred. Resources and time bounds are partially native, conservative passes (full matrix in the technical report).

3 Verification Framework

STEPCHECK separates the artifact being checked from the semantic facts available about it: a shared intermediate representation records the control flow and data movement that ASL exposes, and an annotation slot records the contracts, business states, idempotency, persistence, and compensators that ASL omits. The format-agnostic IR lets analyses run unchanged across input languages; layered annotation keeps raw-ASL verification zero-migration; optional name/resource inference gives existing repositories a non-blocking start; and provenance-based severity keeps CI signals interpretable—native and declared violations error, inferred facts warn.

3.1 Architecture

Every front end lowers to the same workflow graph (Figure 2); every analysis is a pass over it; and the optional emitter lowers verified DSL-authored workflows back to deployable ASL. Raw ASL is the primary input, parsed tolerantly through generic JSON, so intrinsics, context references, SAM placeholders, dialect variants, and JSONata states never cause hard parse failures. The typed DSL emits graph and annotations by construction, and the CNCF front end exercises the same extension point (Section 5.7).

The pass families are summarized in Table 1. Native structural and data-flow checks need only the ASL artifact. *Conservative* native checks report only provable conflicts and may miss others—the opposite polarity to the over-approximating, sound data-flow analysis of Section 3.2—and therefore warn, except the definite misconfiguration SC6003. Semantic checks verify consistency with supplied premises; they do not prove those premises true.

The IR itself is a recursive graph of named states with a designated start. Each state records its ASL kind, normal successors, **Catch** edges, the five-stage I/O pipeline, retry and timeout controls, JSONPath-vs-JSONata mode, nested **Map/Parallel** sub-machines, and the annotation slot. This is enough for every pass; surface syntax stays in the front ends.

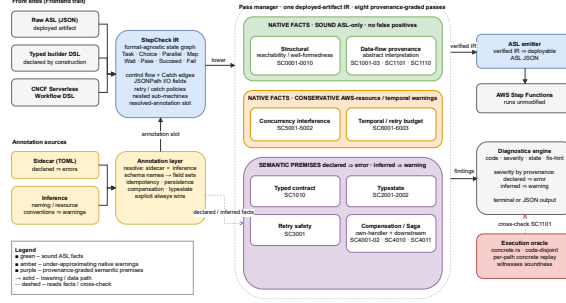


Fig. 2. Verification pipeline: all front ends lower to one deployed-artifact IR shared by the eight passes, rather than running independent lints.

3.2 Native Data-Flow Verification

The native layer is a sound bug finder, not a complete verifier. If STEPCheck reports SC1101, the referenced field is absent on every execution reaching that read. If it is silent, the field may still be absent for some inputs hidden behind $\top$. This is the right direction for a raw-artifact checker facing arbitrary input and opaque task results.

The analysis tracks *document and variable shapes*: either $\top$ (opaque: any field may be present) or a finite record of fields that may occur. Joins union keys, with $\top$ absorbing; a path is definitely absent when record descent reaches a missing key before $\top$. The transfer functions mirror the ASL I/O pipeline: **InputPath**/**OutputPath** re-root the shape, **Parameters**/**ItemSelector** constructors bind static fields and checked $.\$$ reads, opaque task results become $\top$, **ResultPath** merges into raw input preserving siblings, **Assign** updates workflow-variable shapes, and **Catch** edges merge an opaque error object. For JSONata, STEPCheck models direct $\$states.input$, $\$states.result$, and $\$var$ references plus object-valued **Output**/**Assign**/**Arguments** fragments; unsupported expressions project to $\top$ and do not produce SC1101.

A forward iteration joins shapes along normal and **Catch** edges. Over a fixed key universe, a shape only gains keys or lifts to $\top$, so the may-present lattice has finite height. Iteration is bounded by $|states| \times (|keys| + 2) + 2$ rounds. The bounded-depth fragment common in workflow definitions reaches this fixpoint. In general, however, a cycle can re-embed sub-documents and grow shape *depth* past the bound.

To keep soundness independent of that corner case, STEPCheck reports only after convergence. If the round bound is reached, it suppresses SC1101/SC1110 for that machine and its sub-machines rather than emit a partial, under-approximated report. A depth- k widening to $\top$ would be a sound drop-in alternative; see the technical report. In practice the bound is never approached: all 786 data-flow fixpoints over the 193 AWS and 66 CNCF workflows converge within 7 rounds.

Table 1. STEP CHECK’s eight passes. Native sound checks report errors without annotations; conservative checks report warnings except SC6003; semantic checks are relative to supplied facts.

Pass	Codes	What it catches	Claim
Structural and annotation hygiene	SC0001–0012	undefined targets, unreachable/dead-end states, non-exhaustive Choice , stale sidecar references	well-formedness; SC0007 policy warning
Data-flow provenance	SC1001–1003, SC1101, SC1110	missing-field references, binding faults, absent-operand Choice guards (warning)	sound bug finding (Thm. 1)
Concurrency	SC5001–5002	fan-out branches or iterations overwrite the same external resource	conservative warning
Temporal	SC6001–6003	unbounded callback, retry budget past timeout, heartbeat $\geq$ timeout	conservative; SC6003 error
Typed contract	SC1010	successor requires a field its immediate predecessor does not produce	declared-tier precise (Prop. 1)
Typestate	SC2001–2002	out-of-order task composition or illegal protocol edge	declared-tier precise (Prop. 1)
Retry-safety	SC3001	broad retry of a non-idempotent task	relative to idempotency premise
Saga compensation	SC4001–4002, SC4010–4011	missing/incomplete compensation for persistent effects	relative to declared model (Thm. 2)

The restock loop in Figure 1 illustrates why a fixpoint matters. **CheckStock** has one incoming shape from **ReserveInventory** and one from the **WaitRestock** back-edge; joining them keeps any loop-produced field alive, so SC1101 fires on the downstream **ChargeCard** read only because *no* iteration produces **total**.

Theorem 1 (Sound bug reports for missing-field references). *Under the modeled JSONPath/JSONata document-and-variable semantics, if STEP CHECK reports SC1101 for a path p at state s , then every concrete execution reaching s lacks p in the document or variable selected by that read site, so the ASL runtime fails that reference. In the native tier this holds for arbitrary execution input; in the typed tier it holds relative to the closed-world reading of the declared input schema. Expressions outside the modeled fragment lift to $\top$ and are therefore silent.*

The proof is the standard abstract-interpretation argument—each transfer over-approximates its concrete reshaping, so definite absence at the fixpoint

implies absence on every execution—and is given in full in the technical report. SC1110 applies the same definite-absence judgment to **Choice** guards but stays a warning outside Theorem 1: rule reachability depends on earlier guards such as **IsPresent**, so it marks a provably dead or risky branch rather than an unconditional runtime fault.

3.3 Concurrency and Temporal Analyses

The two remaining native passes are conservative engineering checks. **Parallel** branches and concurrent **Map** iterations receive document copies, so only external resources can race. STEP CHECK records statically known resource keys (**TableName**, **QueueUrl**, **Bucket**, and other named targets; full list in the technical report). It emits SC5001 only for provably overwriting operations—**PutItem**/**PutObject** and statically named **states:startExecution** collisions—leaving keyed, append-only, and compute-only operations silent.

The temporal pass checks resilience knobs the platform does not cross-check. SC6001 warns when a callback or Activity task lacks a timeout or heartbeat and can wait to the platform maximum. SC6002 warns when a retry budget can exceed the machine timeout. SC6003 errors on the definite misconfiguration **HeartbeatSeconds** $\geq$ **TimeoutSeconds**.

3.4 Semantic Verification

The semantic layer consumes task facts ASL does not contain: a contract $T : A \rightarrow B$ (fields consumed and produced), a typestate transition $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ in protocol δ , idempotency (whether a retry repeats an external effect), and persistence with an optional compensator. These are premises (Section 3.5); STEP CHECK checks the workflow relative to them.

Proposition 1 (Declared-tier consistency). *For a fully declared workflow W , SC1010/SC2001/SC2002 emit no finding iff every normal-flow task adjacency $T_1 \rightarrow T_2$ satisfies $\text{in}(T_2) \subseteq \text{out}(T_1)$ and $s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2}$, and every task transition belongs to the declared protocol δ . On the series-parallel fragment, this edge condition is equivalent to a standard field/typestate typing judgment. Completeness is relative to the supplied annotations.*

Here $\text{out}(T_1)$ is T_1 ’s declared output set—including fields it forwards, not only those it computes—and $\text{in}(T_2)$ is T_2 ’s declared input set; under this closed-world convention the edge-local check does not false-positive on a field merged upstream and passed through. The checks are edge-local: a ship-before-pay reordering in the order workflow yields typestate mismatches plus field-contract misses. Retry-safety is likewise premise-relative: a broad **States.ALL/States.TaskFailed** retry of a task declared non-idempotent is SC3001, while retries narrowed to specific transient classes are left alone.

Compensation is about downstream error flow. Per-task checks (SC4001/SC4010) can see that a persistent task has no compensating path, but

they miss a later state failing after the effect has committed. For a declared persistent task P with compensator C , SC4011 scans the post-commit region reachable from P before C and asks whether every fallible state’s generic failure routes through a broad `Catch` to C , and whether no normal path reaches `Fail` while bypassing C .

The declared Saga model assumes a single post-commit fault, feasible normal paths, a unique generic catcher for `States.TaskFailed`, and linear handlers whose reachability to C means execution of C . These assumptions exclude failing compensators and branching recovery handlers.

Theorem 2 (Relative soundness and completeness of the compensation check). *For a workflow with declared persistence and compensators, under the declared Saga model, SC4011 emits no finding for persistent task P with compensator C iff every modeled post-commit failure after P executes C . Thus SC4011 is sound and complete relative to the declared model.*

Outside these assumptions, SC4011 remains a reachability check but is no longer sound or complete. The model-independent SC4001/SC4010 checks still flag any persistent effect with no compensating path. Coverage therefore degrades to the per-task tier rather than going silent; the built-in and corpus sagas satisfy the assumptions.

3.5 Recovering Workflow Semantics

Declared facts enter through a sidecar for existing ASL or the typed DSL for greenfield workflows; once declared they support error-level findings, and where absent the declared-tier checks are silent.

Listing 1.1. A sidecar annotation (TOML), excerpt.

```
[tasks.ChargeCard]
idempotent = false
persistent = true
compensation = "RefundPayment"
state_in = "OrderReserved"
state_out = "OrderPaid"
```

The order saga’s complete sidecar is 69 lines of TOML—about eight per-task lines over four tasks, plus shared schema and protocol blocks—and the typed DSL emits the same facts by construction. Optional inference gives existing repositories a zero-effort warning mode: read-like names imply idempotent, write-like non-idempotent, resource-acquiring persistent, and notifications non-idempotent but not persistent; with no signal it abstains, and explicit annotations override it. Stale sidecar and schema references are caught syntactically (SC0011/SC0012). The residual risk, semantic drift between a declared premise and the deployed Lambda, is outside STEP-CHECK’s scope; generating the sidecar from code or IaC keeps declared facts in sync.

4 Implementation

The framework of Section 3 is realised directly as **stepcheck**, a single dependency-light Rust command-line tool of 5,045 lines that follows the pipeline of Figure 2, covering eight analysis passes, three front ends, the ASL emitter, and the self-validation tooling.

Two small extension interfaces realise the architecture: a **Frontend** lowers an input format to the IR, and a **Pass** analyses that IR and emits diagnostics. Behind them sit the typed DSL, raw ASL, and CNCF Serverless Workflow [9]; adding the last touched neither the IR nor any pass (Section 5.7). The raw-ASL frontend lowers generic JSON with a stack-growing deserializer, parsing all 193 corpus workflows and the nested-Map stress cases without error.

Running the verifier. STEP CHECK is a single self-contained binary, packaged for **crates.io** and **npm** (packaging details in the technical report). A developer runs **stepcheck check workflow.json** on an existing definition; **-infer** lights up the semantic checks from name and resource conventions, and **-annot sidecar.toml** promotes them from warnings to declared-tier errors (Section 3.5). Findings render for a terminal or, with **-json**, as a stable machine record (code, severity, state, message, remediation).

By default the exit status is 0 (clean), 1 (any *error*), or 2 (parse failure). The *recommended CI policy* fails a build on errors—the sound native and declared-tier findings—while leaving *inferred warnings* non-blocking. This lets a repository adopt STEP CHECK without triaging heuristic findings up front. **-deny-warnings** tightens the gate once a workflow is annotated. The artifact ships a copy-pasteable GitHub Actions gate.

Nothing else is deployed: the check reads the ingested ASL and never re-emits it, so raw-ASL verification is entirely independent of the emitter.

The remaining components validate the verifier itself. A code-disjoint execution oracle re-interprets a workflow concretely along each acyclic path and cross-checks every data-flow finding—disjointness matters because a shared transfer-function bug would otherwise validate itself—turning the soundness theorem into a tested property (Section 5.4). A mutation command injects one defect of a chosen class and re-emits deployable ASL for the study in Section 5.

Implementation limitations. Distributed **Map** item contexts are modelled coarsely. The ASL emitter—used as the DSL’s code generator and in evaluation harnesses—preserves control flow, retries, catches, and per-state timeouts, but does not yet claim full-fidelity regeneration of arbitrary ingested ASL I/O-processing fields.

5 Evaluation

We evaluate STEP CHECK on real serverless workflows: RQ1, does it detect the defect classes of Figure 1? RQ2, what does verification cost? RQ3, does the shared-IR design extend beyond AWS? Controlled detection power (mutation) is

kept separate from real-world evidence (unmodified workflows and independently fixed bugs).

5.1 Setup and corpus

We mined **193** Step Functions workflows from two public AWS sample collections, taking every parseable ASL definition and excluding build-configuration JSON. The corpus contains 1,355 state entries: 1,354 valid states (median 6, max 33) plus one malformed non-state `QueryLanguage` entry reported as SC0010. It has 166 `Choice`, 64 `Map`, and 33 `Parallel` states; 84 workflows with retries; and 51 with catch handlers. It also includes **5** JSONata-mode, **19** Distributed Map, and **18** nested-`states:startExecution` workflows across nine AWS services. All experiments use a Rust 1.96 release build on a commodity laptop.

5.2 Detection in the wild

Run unmodified over the corpus, with optional inference enabled as a non-blocking adoption mode, STEP CHECK flags **99 of 193** workflows. The native findings are small but concrete. Nine workflows have a `Choice` with no `Default` (SC0007), a runtime-risk warning because an input matching no rule fails. One workflow nests a machine-level `QueryLanguage` directive inside `States` (SC0010). Another 33 callback tasks lack a timeout or heartbeat (SC6001); each can wait up to the platform limit while holding business state open. The sound SC1101, dead-guard SC1110, and concurrency checks fire on none of the 193 (precision on curated examples); their power is exercised below.

The semantic warnings surface adoption targets: SC3001: 36, SC4001: 117, and SC4010: 28—not declared-tier claims until the facts are declared. Against human gold labels (inter-annotator $\kappa = 0.89$ idempotency, 0.99 persistence) these inference-driven warnings are precise where the inferred premise is reliable: among gold-covered findings, SC4001 is **33 of 37 (89%)** true positive, and the effect-aware SC4010 is 6 of 6; SC3001 is **7 of 7 (100%)**. Overall, SC3001/SC4001 contribute **40 of 44 (91%)** true positives. All three inferred semantic warning codes contribute **46 of 50 (92%)**. The downstream declared check SC4011 is silent on the wild corpus because no workflow declares a compensator. On the two built-in declared Sagas (7 persistent steps), it reports zero findings; after a one-edge chain break that redirects `RefundPayment` to failure, it reports exactly `CreateOrder` and `ReserveInventory`, the upstream effects that would leak.

5.3 Detection power versus existing validators

For each applicable workflow, `stepcheck mutate` injects one defect and re-verification counts a detection only when the expected diagnostic increases over the unmutated baseline. Across **616** mutants in six corpus-scale classes, STEP CHECK detects every injected defect (Figure 3a and Table 2). This is per-check reliability on targeted operators, not a claim of complete bug coverage.

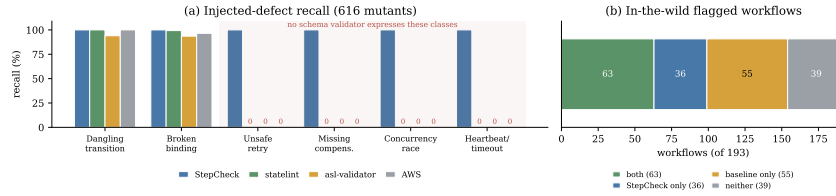


Fig. 3. Detection versus the validator panel: (a) recall over 616 mutants—STEPCHECK detects all six classes, existing validators none of the four semantic ones; (b) mostly disjoint in-the-wild coverage over the 193 workflows.

Table 2. Injected-defect detection over 616 mutants. A mutant counts as detected only when a tool emits a new problem naming the injected fault; incidental schema nits are not credited.

Defect class	Check	Appl.	STEPCHECK	Baseline validators		
				statelint	asl-val.	AWS
Dangling transition	SC0002	166	166 (100%)	166 (100%)	156 (94%)	166 (100%)
Broken data binding	SC1003	141	141 (100%)	140 (99%)	132 (94%)	136 (96%)
Unsafe retry	SC3001	103	103 (100%)	0	0	0
Missing compensation	SC4001	19	19 (100%)	0	0	0
Concurrency interference	SC5001	21	21 (100%)	0	0	0
Temporal heartbeat/timeout	SC6003	166	166 (100%)	0	0	0
Total		616	616 (100%)	306 (50%)	288 (47%)	302 (49%)

We compare against the validators developers already use—`statelint`, `asl-validator`, and AWS’s authoritative `ValidateStateMachineDefinition`. This is a complementarity test, not a superiority claim: those validators catch schema-expressible dangling transitions and broken JSONPath syntax, but do not attempt unsafe retries, missing compensations, races, or heartbeat/timeout violations. In the wild the two flag largely disjoint workflows (Figure 3b): STEPCHECK alone flags 36, the panel alone 55, mostly schema nits.

5.4 Data-flow provenance: soundness, power, and reach

The central SC1101 claim is no false positives. It fires zero times on the 193 unmodified workflows. To exercise its power, the typed-tier mutator is seeded with declared input fields and injects a missing-field read into each of the **126** workflows with a suitable site. STEPCHECK detects **84 of 126 (67%)**. An independent concrete execution oracle, implemented separately from the analysis, confirms every one of the 84 absent on all acyclic reaching paths, with zero counterexamples.

The check is not vacuous: it catches the `amount/total` miss validators accept, and flow-sensitive misses through `ResultPath` merges that adjacent-task field-subset checks miss. The undetected third are downstream of opaque task results, where the shape soundly collapsed to $\top$.

Table 3. Confirmed in-the-wild defects re-discovered on deployed artifacts.

Code	Repository	Defect fixed
SC1101	aws-batch-runtime-monitoring	document field used instead of context input
SC1101	serverless-ai-fitness	tasks read fields never produced
SC1110	serverless-ai-fitness	subscriber branch unreachable
SC6001	leave-management	callback can hang indefinitely
SC0002	airway-shipment-orchestrator	transition targets missing state
SC0003/4/5	video-translation	string End leaves dead states

The modeled fragment covers most real reads: of the corpus’s genuine document-field references, **92%** use dotted access and resolve precisely, and **8%** use bracket/wildcard/filter syntax abstracted to $\top$ ($\$ \$$ context and **States.*** operands are not document references; **stepcheck path-coverage**, full breakdown in the technical report).

5.5 Real defects in the wild

We mined fix-commit pairs whose diff edits an ASL definition and replayed **STEPCHECK** across each fix. On **7 of 39** pairs a pre-fix finding disappears after the fix; adjudication confirms **6** genuine across five repositories, one an official AWS sample (Table 3); four are semantic/logic classes no schema validator expresses. This is existence evidence, not a prevalence estimate.

5.6 Cost, overhead, and deployment

Verification is cheap: the eight passes average $\approx 39 \mu s$ per workflow (median $\approx 25 \mu s$, max ≈ 1.1 ms), the whole corpus verifies in ≈ 7.5 ms, and the check adds no runtime component. Synthetic acyclic chains scale near-linearly: 100 states in ≈ 0.2 ms, 1,000 in ≈ 2.6 ms, 10,000 in ≈ 31 ms, 30,000 in ≈ 0.13 s. A topology stress sweep exercises harder shapes: a 512-branch **Parallel** fan-out (2,051 states) verifies in ≈ 49 ms, a 160-level nested **Map** (161 states) in ≈ 11 ms. The DSL stays concise (20 declarations expand to a 119-line annotated ASL definition), and a DSL-verified order workflow deployed unmodified as an AWS Express machine reached **SUCCEEDED**.

5.7 Cross-format generalization: CNCF Serverless Workflow

The CNCF front end parses **66** Serverless Workflow examples and lowers them to the same IR without changing any pass. Structural, typestate, retry, compensation, concurrency, and temporal checks apply unchanged; the ASL-specific JSONPath data-flow checks stay silent because CNCF uses runtime expressions. Mutation

detects every injected defect with a site (retry 4/4, structural 9/9, concurrency 1/1, temporal 50/50), and where CNCF declares JSON Schemas SC1010 uses them exactly: a reordered schema-annotated order workflow is rejected with three contract errors.

5.8 External validity: independent repositories

To test whether the analyses overfit AWS’s own samples, we ran STEPCHECK unmodified on **95** further ASL definitions from **16** public repositories outside the AWS sample collections (50 domain workflows across 13 repositories, the rest validator fixtures; script and manifest in the artifact). The sound SC1101 fires on a credit-card transaction-reversal workflow: its **Pass** state reconstructs the document through **Parameters** into a closed record that drops the **ledgerEntries** field a downstream **Map** iterates. This is a flow-sensitive miss no adjacent-field check sees. The concurrency check flags an IoT firmware-upgrade workflow whose three **Parallel** branches write one DynamoDB table. Declaring the two durable effects of an independent payment-settlement pipeline through a sidecar promotes STEPHECK’s finding to a declared-tier *error*: a settlement failure leaves an authorized hold that is never voided. These hits, on workflows we neither authored nor curated, show the checks generalize beyond the sample corpus.

Threats to validity. STEPHECK verifies workflow definitions, not opaque Lambda or managed-service internals. Opaque results and JSONata expressions outside the modeled direct-reference/object fragment lift to $\top$, preserving soundness but losing precision. The mutation operators are our own, so the 100% figure is per-operator reliability against a targeted fault population, not detection power against an independent one. The in-the-wild and real-fix results (Sections 5.2, 5.5, 5.8) supply the complementary uncontrolled evidence. Declared-tier semantic results are relative to supplied annotations, whose truth is outside STEPHECK’s scope. The execution oracle cross-checks SC1101 along acyclic reaching paths only, so the loop-join fixpoint step rests on the convergence argument (Section 3.2). The main corpus is public sample repositories and skews small; proprietary industrial definitions are excluded, so the generated sweeps exercise scale and topology but not industrial semantic diversity (extended limitations in the technical report). The concurrency pass checks effects inside a child machine only when that child is itself checked. The deployment smoke test uses one Express-mode emission, not Standard mode or a live callback.

6 Related Work

Runtime orchestration and durable execution. Triggerflow rethinks how serverless workflows run [5], while durable-execution runtimes—Azure Durable Functions [8], Temporal [18], and Beldi’s transactional guarantees for stateful serverless workflows [20]—enforce recovery and transactions *during* execution. STEPHECK is complementary: it statically checks the deployed Step Functions

artifact *before* deployment with no new runtime, pursuing the same failure-atomicity goal from the opposite end.

Formal verification at other layers. AWS uses TLA⁺ to model-check service designs, workflow soundness has been studied with Petri nets and service-composition model checking [15,1], and Zelkova brings SMT reasoning to IAM access policies [6]. These verify an abstract model, the platform, or a different artifact; STEP CHECK checks the executable ASL definition with linear passes suitable for CI. Closest to our retry-safety check, Flux verifies idempotence of stateful serverless functions [11] and could *supply* the per-task idempotency premises STEP CHECK’s SC3001 consumes.

Behavioural types and Sagas. Typestate and session types provide the basis for protocol conformance [16,13], and the Saga pattern is the standard compensation model [12]; STEP CHECK adapts both to serverless task graphs as edge-local declared checks and a declared-model reachability check.

Data-flow and cloud validators. Existing Step Functions validators check schema, path syntax, or execute tests, but not field provenance [4,7,2,3]. Data-flow anomalies have been studied for BPMN and BPEL [17,19], but there data is abstracted to named elements in a hand-built model; STEP CHECK tracks nested JSON document shapes through ASL’s I/O pipeline as an abstract interpretation [10]. Serverless formal foundations motivate the setting but do not provide static workflow verification [14].

7 Conclusion

STEP CHECK shows that useful static verification can run directly on the AWS Step Functions artifact developers already deploy. Its native data-flow analysis reports only definite missing-field failures. Its layered annotations make retry, typestate, and Saga checks precise relative to supplied facts. On real workflows it rediscovers independently fixed bugs, detects targeted semantic mutants missed by existing validators, and runs fast enough for CI.

The sound core and the higher-stakes semantic checks sit in different tiers by design. Missing-field faults are caught with certainty on the raw artifact; the rediscovered and in-the-wild SC1101 bugs show how they escape testing behind **Choice** branches. Retry, race, and compensation faults instead begin as provenance-graded warnings and harden into declared-tier errors as effects are annotated or inferred. The layering is a migration path, not a fixed line. Serverless orchestration need not wait until execution to expose data, ordering, retry, timing, and compensation mistakes.

Artifact availability. An anonymized artifact—STEP CHECK, the corpus, scripts, figures, and reproduction instructions—is at <https://anonymous.4open.science/r/StepCheck>. The paper is self-contained; the artifact adds a companion technical report with the full formal development, algorithms, feature matrices, and extended limitations.

References

1. van der Aalst, W.M.P.: The application of Petri nets to workflow management. *Journal of Circuits, Systems and Computers* **8**(1), 21–66 (1998)
2. Amazon Web Services: ValidateStateMachineDefinition — AWS Step Functions API reference. https://docs.aws.amazon.com/step-functions/latest/apireference/API_ValidateStateMachineDefinition.html (2024)
3. Amazon Web Services: AWS step functions developer guide (amazon states language). <https://docs.aws.amazon.com/step-functions/> (2026)
4. Amazon Web Services Labs: statelint: A validator for Amazon states language documents. <https://github.com/aws-labs/statelint> (2024)
5. Arjona, A., García López, P., Sampé, J., Slominski, A., Villard, L.: Triggerflow: Trigger-based orchestration of serverless workflows. *Future Generation Computer Systems* (2021)
6. Backes, J., Bolignano, P., Cook, B., Dodge, C., Gacek, A., Luckow, K., Rungta, N., Tkachuk, O., Varming, C.: Semantic-based automated reasoning for AWS access policies using SMT. In: *Formal Methods in Computer-Aided Design (FMCAD)*. pp. 1–9 (2018)
7. Bougère, C.: asl-validator: A JSON-schema validator for Amazon states language documents. <https://github.com/ChristopheBougere/asl-validator> (2024)
8. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C.S.: Durable functions: Semantics for stateful serverless. *Proc. ACM Program. Lang.* **5**(OOPSLA) (2021)
9. Cloud Native Computing Foundation: Serverless workflow specification. <https://serverlessworkflow.io/> (2026)
10. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. pp. 238–252. ACM (1977)
11. Ding, H., Wang, Z., Shen, Z., Chen, R., Chen, H.: Automated verification of idempotence for stateful serverless applications. In: *USENIX Symp. on Operating Systems Design and Implementation (OSDI)* (2023)
12. Garcia-Molina, H., Salem, K.: Sagas. In: *ACM SIGMOD Int. Conf. on Management of Data*. pp. 249–259 (1987)
13. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: *ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*. pp. 273–284 (2008)
14. Jangda, A., Pinckney, D., Brun, Y., Guha, A.: Formal foundations of serverless computing. *Proc. ACM Program. Lang.* **3**(OOPSLA), 1–26 (2019)
15. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How Amazon web services uses formal methods. *Communications of the ACM* **58**(4), 66–73 (2015)
16. Strom, R.E., Yemini, S.: Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* **SE-12**(1), 157–171 (1986)
17. Sun, S.X., Zhao, J.L., Nunamaker, J.F., Sheng, O.R.L.: Formulating the data-flow perspective for business process management. *Information Systems Research* **17**(4), 374–391 (2006)
18. Temporal Technologies: Temporal: Durable execution and workflow orchestration. <https://temporal.io/> (2026)

19. Trcka, N., van der Aalst, W.M.P., Sidorova, N.: Data-flow anti-patterns: Discovering data-flow errors in workflows. In: Int. Conf. on Advanced Information Systems Engineering (CAiSE). LNCS, vol. 5565, pp. 425–439 (2009)
20. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: USENIX Symp. on Operating Systems Design and Implementation (OSDI) (2020)